
SILENT FAILURES AT THE 2^{32} BOUNDARY: A TECHNICAL REPORT ON LARGE-TENSOR MATRIX MULTIPLICATION IN PYTORCH’S APPLE MPS BACKEND

TECHNICAL REPORT

 **Junichiro Niimi***
Meijo University

ABSTRACT

Apple Silicon machines with 192 GB or more of unified memory make it routine to place tensors with more than 2^{32} elements on a desktop GPU. We show that PyTorch’s Metal Performance Shaders (MPS) backend silently returns wrong results for batched matrix multiplication at this scale. On macOS 27.0, `torch.bmm`, and therefore `torch.matmul` and eager attention, returns relative errors above 1 without an exception or a warning, in every PyTorch release from 2.4.1 to 2.14.0 that we tested. On one machine, we sweep `bmm` over two dtypes, four memory layouts, six shapes and 42 batch sizes between 4096 and 65538 (1584 runs on PyTorch 2.14.0, and a reduced sweep on ten earlier releases), and judge every result against a float64 computation on the CPU. Three rules account for every outcome on 2.14.0. When the output exceeds 2^{32} elements and an operand is a transposed view, the entire output is wrong and equals a computation that ignores the strides of that operand. When a contiguous input exceeds 2^{32} elements, only the batches beyond that point are wrong, and they equal a computation whose index wraps around at 2^{32} . Operands that are views with at least 2^{31} elements raise an exception instead, so a larger problem can turn an explicit error into a silent failure. A control on CUDA is correct for `bmm`, although `torch.arange` is silently wrong above 2^{32} elements there as well. In a public sentiment classifier, one oversized batch corrupts a third of the outputs and collapses them onto a single class. The study is black-box: we report what the backend returns, compared with reference results. We release the sweep harness, the raw results and a guard that stops any MPS operation touching 2^{32} or more elements at <https://github.com/jniimi/mps-silent-failures>.

1 Introduction

With unified memory, the CPU and the GPU of a machine share a single pool of memory, so a model and its data can be placed and moved without the fixed capacity of a separate GPU memory. This flexibility matters more as models, contexts and batches grow, and it has made a desktop machine a practical place to build and analyze large models. Apple Silicon machines offer 192 GB or more of such memory, and a single fp32 tensor with 2^{32} elements (17 GB) fits with room to spare. Ordinary workloads reach this size: attention over long contexts or large batches, feature extraction for probing, and attention-map analysis. For example, attention scores of shape $(B \cdot H, L, L)$ pass 2^{32} elements at 32 heads, a context of 4096 tokens and a batch of 9.

PyTorch [1] runs on Apple GPUs through its MPS backend. On macOS 14, batched matrix multiplication whose output exceeded 2^{32} elements stopped with an explicit error (Tiling of batch matmul for larger than 2^{32} entries only available from MacOS15 onwards). After an upgrade of the same machine to macOS 27.0, the same operation runs to completion but returns wrong values, without any error or warning [2], and we find further silent failures nearby. For this operation, upgrading the operating system turned a hard failure into a silent one.

*jniimi@meijo-u.ac.jp

Silent failures are among the most costly framework bugs, because nothing in the pipeline flags them [3]. In the case study of Section 4.2, overall accuracy drops by only 7 points while recall for two of the three classes falls to almost zero on the affected inputs, and no error, warning or non-finite value appears.

Contributions Our contributions are as follows:

- A boundary-targeted differential sweep of `bmm` on the MPS backend, on one machine with macOS 27.0. It varies the dtype (fp32, fp16), the memory layout (contiguous, either operand transposed, sliced with a storage offset), the shape, and whether the output or an input crosses a candidate boundary (2^{31} elements, 2^{32} elements, 2^{32} bytes), and the driver can bisect between grid points to locate a switch, although no bisection was needed. Results are judged against a float64 computation on the CPU, over all elements at the switch points and over a sample of batches elsewhere. The full sweep uses PyTorch 2.14.0, and a reduced sweep is repeated for ten earlier releases.
- Index-encoded inputs that let us read off, from the outside, which input batch, or which row and column within a batch, a wrong output element was computed from.
- A first-pass suite of 26 test cases (`bmm`, attention, convolution, and elementwise, reduction and indexing operations on large tensors), run on 11 PyTorch releases, that records for each case whether it is correct, raises, crashes, or is silently wrong. It includes a regression in 2.14.0 where `torch.arange` stops raising and silently truncates. The `bmm` and attention cases are repeated on CUDA as a control.
- A case study on a public sentiment classifier, in which a single oversized batch corrupts a third of the outputs without any error, the affected predictions collapse onto one class, and chunked execution of the same inputs is correct.
- A drop-in guard that stops any MPS operation that would create or read a tensor with 2^{32} or more elements, its overhead, and its evaluation against the sweep.

2 Related Work

Bugs in deep learning frameworks. Empirical studies classify the bugs reported against deep learning frameworks by symptom and root cause [4]. Tamboon et al. [3] study *silent* bugs in Keras and TensorFlow, which produce wrong behavior without a crash or an error message, and find that they are hard for users to notice and to trace back to the framework. Wang et al. [5] study numerical bugs in deep learning programs, such as overflow and loss of precision in floating-point computation. The failures we report are silent in the sense of that study [3], but they are not rounding problems: the relative error is of order one and appears only above a fixed element count. These studies mine issue trackers after the fact, whereas we start from a single reported issue [2] and measure systematically where the failure occurs.

Testing deep learning libraries. Without a ground-truth oracle, most library testing is differential. CRADLE [6] runs the same model on several Keras backends and flags inconsistent outputs. Fuzzers generate the test inputs automatically: FreeFuzz [7] mines API calls from documentation, tests and models, and compares CPU with GPU execution; NNSmith [8] and generation-based differential fuzzing [9] build valid computation graphs; TitanFuzz [10] lets a large language model write the test programs. Our sweep is differential as well, comparing MPS with a float64 result on the CPU (CUDA serves only as a control), and the case study in Section 4.2 uses a metamorphic relation [11] between one large batch and its chunks. The difference lies in what is varied. The tools above aim at breadth over APIs and operator combinations and need each test to be cheap, while a single fp32 tensor with 2^{32} elements already takes 17 GB. To our knowledge, none of them targets element-count boundaries, and none uses the MPS backend as the system under test. We fix a small family of operations and vary size, memory layout and library version instead.

Reproducibility across hardware and silent data corruption. Training results vary between runs and between accelerators because of nondeterministic kernels and differences in floating-point reduction order [12, 13]. Such variation is small per operation and is expected behavior, yet it can grow over training: Pham et al. [12] report that implementation-level nondeterminism alone changes overall accuracy across identical training runs by up to 2.9%, and per-class accuracy by up to 52.4%. Zhuang et al. [13] likewise find that randomness introduced by hardware and software tooling during training leaves top-line accuracy almost unchanged but has a far larger effect on per-class metrics and on subgroups of the data. At the other extreme, studies of large fleets report silent data corruption caused by defective processors, which affects individual machines and is often intermittent [14, 15]. The failures in this report belong to neither group. They are deterministic, they reproduce from a few lines of code across 11 PyTorch releases, and in our measurements whether they occur is decided by tensor size, memory layout and library version, so they are software faults rather than noise or hardware defects.

Position of this report. In summary, bug studies classify what has already been reported, testing tools compare backends or generate programs with the aim of covering many APIs cheaply, and work on reproducibility concerns small numerical variation or defective hardware. We instead target a size boundary directly, on a backend that these lines of work have not examined. This is a technical report on observable behavior and does not propose a new testing method. Its contribution is a reproducible record of where PyTorch on Apple GPUs returns wrong results for large tensors, which users of large-memory Apple Silicon machines can reach in ordinary workloads.

3 Method

From matmul to bmm. `torch.matmul` (the `@` operator) dispatches on the number of dimensions. For inputs with three or more dimensions it broadcasts the leading batch dimensions, folds them into one, and calls `torch.bmm` on (B, n, m) and (B, m, p) tensors. The attention mechanism originates in neural machine translation [16]. The form used in current models is the scaled dot-product attention of the Transformer [17], $\text{softmax}(QK^\top/\sqrt{d})V$. The Hugging Face Transformers library [18] calls the implementation that evaluates this formula step by step *eager* attention (`attn_implementation="eager"`),¹ in contrast to fused kernels that never return the score matrix. Eager attention is therefore the textbook computation and not a special variant. It computes $QK^\top$ and then $\text{softmax}(QK^\top)V$ with `matmul`, so both products go through `bmm`, and the scores of shape (B, H, L, L) exist as a tensor. `nn.Linear` instead flattens its input to two dimensions and calls `addmm`, and fused `scaled_dot_product_attention` (SDPA) is a single operator that does not return the score matrix to the caller.

Environment. All MPS results in this version come from one machine (Table 1). PyTorch versions are installed from the official PyPI wheels into fresh environments with `uv`, one environment per version.

Table 1: Test machine for the MPS results.

Model	Mac Studio (model identifier Mac14,14)
Chip	Apple M2 Ultra: 24-core CPU (16 performance, 8 efficiency), 76-core GPU
Memory	192 GB unified
OS	macOS 27.0 (build 26A428)
Firmware	20457.1.29
Python	3.12.12
PyTorch	2.4.1, 2.5.1, 2.6.0, 2.7.1, 2.8.0, 2.9.1, 2.10.0, 2.11.0, 2.12.1, 2.13.0, 2.14.0

Axes. The sweep targets one operation, `torch.bmm` on a of shape (B, M, K) and b of shape (B, K, N) , and varies the axes in Table 2. Every shape has 2^{16} elements per batch in the tensor that is meant to cross the boundary (the *crossing tensor*), which is the output ($MN = 2^{16}$) in the output-side shapes and the input a ($MK = 2^{16}$) in the input-side shapes. That tensor therefore has $2^{16}B$ elements. In the base shapes the other two tensors have $2^{14}B$ elements each and stay near 2^{30} . The additional shapes split the same number of elements into different (M, K, N) , to check that the behavior depends on the element count and not on a particular dimension. In each of them one of the other tensors has $2^{15}B$ elements and reaches 2^{31} at $B = 65536$. The layouts change only how the operands are stored. The logical values of a and b are generated on the CPU from a fixed seed per batch, independently of the layout, and the storage is built from them, so that for the same seed all four layouts compute the same matrix product and their results can be compared directly. Operations other than `bmm` are covered only by the first-pass suite (Section 4.3).

Boundaries and search. We consider three candidate boundaries for the crossing tensor: 2^{31} elements ($B_0 = 32768$), 2^{32} elements ($B_0 = 65536$), and 2^{32} bytes ($B_0 = 16384$ in fp32; in fp16 this coincides with 2^{31} elements). These candidates are hypotheses about where a limit could lie. A 32-bit signed integer can index at most $2^{31} - 1$ elements and an unsigned one at most $2^{32} - 1$, so an implementation that holds an element index in either type would change its behavior at the first or the second candidate. The third candidate tests whether a limit applies to the number of bytes instead of the number of elements: in fp32 a tensor reaches 2^{32} bytes at 2^{30} elements, and a byte limit would also make fp16 and fp32 switch at different batch sizes. Each series, that is, each combination of shape, dtype and layout, is run at the five consecutive batch sizes $B_0 - 2, \dots, B_0 + 2$ around every candidate, at a small baseline point ($B = 4096$, 2^{28} elements), and on an evenly spaced grid between these points (seven values per interval for the base shapes and three for the additional ones). This gives 37 batch sizes per series in fp32 and 25 in fp16 for the base shapes. A change of behavior that does not coincide with a candidate would appear as two neighboring grid points

¹https://huggingface.co/docs/transformers/attention_interface

Table 2: Axes of the bmm sweep. The base shapes are run in both dtypes and with all input kinds, the additional shapes in fp32 with random inputs.

axis	values
shape (M, K, N)	output-side: base=(256, 64, 256), (512, 64, 128), (128, 64, 512) input-side: base=(256, 256, 64), (512, 128, 64), (128, 512, 64)
dtype	fp32, fp16
layout	contiguous; b transposed; a transposed (a contiguous storage of the transposed values, passed as a <code>transpose(1, 2)</code> view); sliced (both operands are <code>[1:]</code> of a storage with one extra leading batch, so the storage offset is one batch)
input values	random (uniform on $[-1, 1]$, zeros replaced so that every element is non-zero); index-encoded (see below)
batch size B	4096 to 65538 (see below)
PyTorch	2.14.0 for the full sweep; 2.4.1 to 2.13.0 for a reduced sweep (base shapes only)

with different outcomes, and the driver then bisects between them down to a single batch. At each switch point, the batch sizes on both sides are run again with a comparison over all elements and three seeds; a series without a switch receives this comparison at its largest batch size.

Reference and comparison. The reference is the same product computed in float64 on the CPU. To build it, the logical inputs of each batch are generated again on the CPU from their seeds; inputs are never read back from the device for this purpose, so a fault in the transfer cannot cancel out. The device output is copied to the CPU in chunks, compared, and discarded. Let Y_i be batch i of the device output and Y_i^* the same batch of the reference, and let $\|A\|_{\max}$ denote the largest absolute entry of a matrix A . The error of batch i is

$$e_i = \frac{\|Y_i - Y_i^*\|_{\max}}{\|Y_i^*\|_{\max}}, \quad (1)$$

and batch i is wrong when $e_i > \tau_d$, where the tolerance τ_d depends only on the dtype d . Let $\hat{e}_d$ be the largest e_i in a set of calibration runs far below any candidate (base shapes, $B = 4096$, contiguous, three seeds, all elements). Then

$$\tau_d = \max(10 \hat{e}_d, \tau_d^{\min}), \quad (2)$$

with a floor $\tau_d^{\min}$ of 10^{-6} in fp32 and 10^{-3} in fp16.

A comparison is either *full*, over all batches, or *sampled*. A sampled comparison takes 256 evenly spaced batches, the first 4 and the last 8, and the batches around every position where the flat index of an operand or of the output passes a multiple of a candidate boundary, which gives 270 to 290 batches. The grid and the points around the candidates use sampled comparisons, and the switch points use full comparisons with three seeds. The reduced sweep for earlier PyTorch versions uses a coarser grid (two values per interval) and one seed for the full comparisons.

Each run is classified as *ok*; *error*, when an exception is raised; *crash*, when the process ends without returning a result; *timeout* (20 minutes for a sampled and 60 for a full comparison); *wrong*; *truncated* (an output of zeros), when at least 90% of the wrong elements are exactly zero, which the non-zero inputs make detectable; or *input corrupt*, when the inputs read back from the device in the sampled batches differ from the generated values, which would point to the transfer and not to bmm. Runs whose estimated memory exceeds 60% of the physical memory are skipped. For wrong runs we also test a small set of hypotheses about how the inputs were read. For up to 16 wrong batches at each end of the wrong range, we recompute on the CPU the output that would result if the strides of a transposed operand were ignored, if the flat index into an operand wrapped around at 2^{32} elements, if both happened, or if the storage offset were ignored, and record the fraction of wrong elements that each hypothesis reproduces.

The CUDA runs (Section 4.3) use an NVIDIA A100 with TF32 disabled. They serve as a control and not as an oracle.

Index-encoded inputs. Random inputs show that an output is wrong, but not which input elements it was computed from. Index-encoded inputs make every output element carry that information. One operand is *encoded*: each of its elements holds the code

$$c(f) = (f \bmod P) + 1 \quad (3)$$

of a field f , which is either the batch index i (the same code in the whole batch) or the position $rC + c$ of element (r, c) within its $R \times C$ matrix (the same codes in every batch). The other operand is a *selection* matrix with a single

one in each row or column, identical in every batch. When b is encoded, $a_{i,m,k} = [k = m \bmod K]$, and when a is encoded, $b_{i,k,n} = [k = n \bmod K]$, so that a correct product is

$$y_{i,m,n} = b_{i, m \bmod K, n} \quad \text{or} \quad y_{i,m,n} = a_{i, m, n \bmod K}, \quad (4)$$

respectively. Each output element is then a sum with one non-zero term and equals one code exactly, without rounding. The period P is the largest prime below 2^{20} in fp32 ($P = 1048573$) and below 2^{11} in fp16 ($P = 2039$), so every code is an integer that the dtype represents exactly. From a wrong output element y we decode the field that was actually read, $f' = y - 1$, and record the shift

$$\delta = f' - f \pmod{P}, \quad -P/2 \leq \delta < P/2, \quad (5)$$

against the field f that a correct product would show at that place. In fp32 the decoding is unique, because both the batch sizes and the 2^{16} positions within a matrix are below P ; in fp16 it is known only modulo 2039. With the position encoded, f' gives the row and column that were read in place of the expected ones. The two operands and the two fields give four kinds of input, which we run for the base shapes at $B_0 \pm 1$ around every candidate and at $B = 4096$ with sampled comparisons, and at the switch points with full comparisons.

Isolation. Each run executes in a fresh process, because a failure may abort the process, and a wrong result may corrupt state for later calls in the same process. A crash is recorded as an outcome. The driver appends one JSON line per run to a local file, with the configuration, the environment, the outcome, the error statistics and the timings. A manifest per sweep records the operating system build, the hardware, the Python and PyTorch versions, the tolerances and the SHA-256 hashes of the driver and the worker script. These files are released with the code.

4 Results

4.1 Boundary sweep of `bmm`

The sweep on PyTorch 2.14.0 comprises 1584 runs over 42 batch sizes between 4096 and 65538: 1273 are correct, 214 raise an error, and 97 are silently wrong. No run crashed or timed out, and no wrong output contained a non-finite value. Sampled and full comparisons agree wherever both were run, and so do the three seeds. The largest calibration errors are $\hat{e}_{\text{fp32}} = 1.2 \times 10^{-6}$ and $\hat{e}_{\text{fp16}} = 4.2 \times 10^{-4}$, so the tolerances of Eq. (2) are $\tau_{\text{fp32}} = 1.2 \times 10^{-5}$ and $\tau_{\text{fp16}} = 4.2 \times 10^{-3}$.

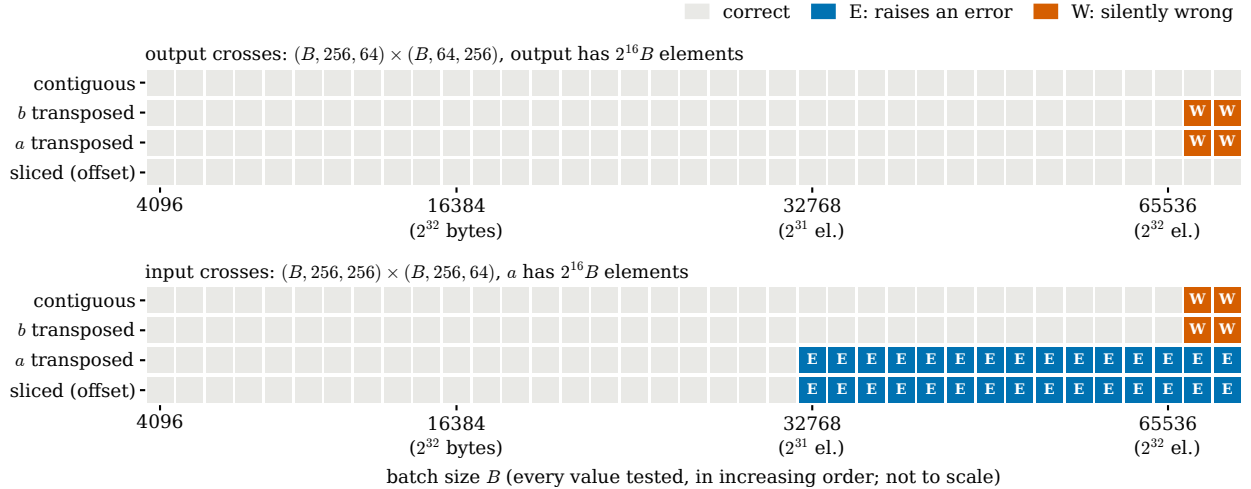


Figure 1: Boundary map of `bmm` on MPS (PyTorch 2.14.0, macOS 27.0, fp32, random inputs). Each cell is one batch size; five consecutive values are tested around each candidate boundary, and the remaining values form a grid. The fp16 results have the same pattern at the same batch sizes.

Boundary map. Figure 1 shows the two base shapes. Nothing changes at 2^{32} bytes. The silent failures start between $B = 65536$ and $B = 65537$, that is, the result is still correct at exactly 2^{32} elements and wrong one batch later. fp16 switches at the same batch sizes as fp32, so the relevant quantity is the number of elements and not the

number of bytes. The exceptions start at $B = 32768$, where the view operand has exactly 2^{31} elements, and carry the message `MPSGraph does not support tensor dims larger than INT_MAX`. Every change of behavior lies between two consecutive batch sizes next to a candidate, so no bisection was triggered and the switch points are exact.

The outcome of every run with random inputs (1088 runs, six shapes, two dtypes, four layouts) is reproduced by the rules in Table 3, applied in order. The order matters. With a of shape $(B, 512, 64)$ passed as a transposed view, the operation is correct at $B = 65535$, raises at $B = 65536$, where a reaches 2^{31} elements while the output has exactly 2^{32} , and is silently wrong at $B = 65537$, where the output exceeds 2^{32} . With sliced operands of the same shape it raises at $B = 65536$ and is correct again at $B = 65537$. A larger problem can therefore turn an explicit error into a silent failure. The rules describe what we observed on this version and are not a specification; in particular, no configuration in the sweep has an output above 2^{32} elements together with an input above 2^{32} .

Table 3: Rules that reproduce all 1088 outcomes of the sweep with random inputs (PyTorch 2.14.0), applied in order. A *view* is an operand passed as a transposed or a sliced tensor.

	condition	outcome	extent
1	output $> 2^{32}$ elements, an operand is transposed output $> 2^{32}$ elements, otherwise	silently wrong correct	every batch of the output
2	an operand that is a view has $\geq 2^{31}$ elements	raises an error	
3	a contiguous operand has $> 2^{32}$ elements otherwise	silently wrong correct	only batches beyond element 2^{32}

Two kinds of wrong result. The two silent failures differ in extent. When the output crosses the boundary and an operand is transposed (rule 1), all 65537 batches are wrong, including batch 0, and the largest error e_i of Eq. (1) is between 1.7 and 2.2. Exceeding the boundary by a single batch thus invalidates the whole output. When a contiguous input crosses the boundary (rule 3), only the batches that lie beyond element 2^{32} of that input are wrong, with e_i between 1.4 and 1.7, and all earlier batches pass the comparison. This is the pattern of the case study in Section 4.2, where the texts before position 1365 are unaffected.

What the wrong values are. For each wrong run we tested the four hypotheses of Section 3 about how the inputs were read, and counted the wrong elements that each reproduces. Under rule 1, every wrong element equals the value obtained when the strides of the transposed operand are ignored, that is, when its storage is read as if it were contiguous. Under rule 3, every wrong element equals the value obtained when the flat index into the large operand wraps around at 2^{32} elements, so that batch 65536 is computed from batch 0. The index-encoded inputs confirm both readings element by element without reference to a hypothesis. With the batch index encoded in a , the shift of Eq. (5) is $\delta = -65536$ for every wrong element: the decoded batch lies exactly 2^{16} batches, that is 2^{32} elements, before the expected one. In fp16 the shift is $\delta = -288$, which is -65536 modulo $P = 2039$. With the position inside the matrix encoded in a transposed b of logical shape 64×256 , the output that should contain element $(0, 1)$ contains element $(1, 0)$, and the one that should contain $(16, 0)$ contains $(0, 64)$, as expected when a 256×64 storage is read row by row as 64×256 . The three rules predict the outcome of 460 of the 496 runs with index-encoded inputs. The other 36 are predicted to be wrong but are correct, and they are explained by the inputs themselves. Some index-encoded inputs do not change under the misreading, for example values that are constant within a batch when strides are ignored, or operands that are identical in every batch when the index wraps by whole batches. We checked this on the CPU for all 64 index-encoded runs in which the rules predict a wrong result, by computing the product under the predicted misreading. It equals the correct product exactly in the 36 runs that were recorded as correct, and differs from it in the 28 runs that were recorded as wrong. With this taken into account, the rules account for all 1584 runs.

Earlier PyTorch versions. Table 4 summarizes the reduced sweep of the base shapes on ten earlier releases. The releases fall into four groups with identical outcomes within each group. Two results hold in every release since 2.5.1: a transposed operand with an output above 2^{32} elements gives a wrong output that matches the ignored-strides computation, and a contiguous input above 2^{32} elements gives wrong batches that match the wrapped index. What changed over time is how an oversized *view* is handled. In 2.4.1 it is accepted below 2^{32} elements and returns an output of zeros from 2^{32} elements on (the truncated outcome; 16 runs, all on this version). From 2.5.1 to 2.8.0 it aborts the process with a Metal assertion (`NDArray dimension length > INT_MAX`) between 2^{31} and 2^{32} elements, raises an exception at exactly 2^{32} elements, and above 2^{32} elements is silently wrong with the wrapped index. From 2.9.1 on it raises an exception everywhere from 2^{31} elements upward. For a sliced operand the thresholds up to 2.9.1 apply to the storage, which holds one batch more than the view, so they are reached one batch earlier. Version 2.4.1 differs in two further ways. With an fp32 output above 2^{32} elements it is wrong for every layout, including contiguous operands,

but only in the first and the last batch, and half of the wrong elements are zero; none of our hypotheses reproduces these values. With fp16 the same configurations are correct. Across all versions no wrong output contains a non-finite value, no run is classified as input corrupt or timeout, no bisection was triggered, and within 2.10.0 to 2.14.0 we found no difference for bmm.

Table 4: Outcome of bmm by PyTorch version (base shapes, fp32 and fp16 unless noted, macOS 27.0). n is the number of elements of the tensor in question. **W**: silently wrong; **Z**: output of zeros, silently; **ok**: correct; **err**: raises; **crash**: process aborts.

condition	2.4.1	2.5.1–2.8.0	2.9.1	2.10.0–2.14.0
output $n > 2^{32}$, contiguous or sliced	W (fp32), ok (fp16)	ok	ok	ok
output $n > 2^{32}$, an operand transposed	W (fp32), ok (fp16)	W	W	W
contiguous input $n > 2^{32}$	W	W	W	W
view input, $2^{31} \leq n < 2^{32}$	ok	crash	err	err
view input, $n = 2^{32}$	Z	err	err	err
view input, $n > 2^{32}$	Z	W	err	err

4.2 Case study: sentiment classification with eager attention

To check that the failure reaches real workloads, we ran a public sentiment classifier through the standard Hugging Face implementation. The model is `cardiffnlp/twitter-roberta-base-sentiment-latest` (revision 3216a57f), a RoBERTa-base encoder [19] with 12 attention heads that was pretrained on tweets and fine-tuned for three-class sentiment analysis in the TimeLMs project [20]. We run it with `attn_implementation="eager"`, in fp32 under `torch.inference_mode()`. The inputs are the first 2048 tweets of the sentiment test split of the TweetEval benchmark [21] (revision b3a375ba), tokenized once with padding to $L = 512$ and reused unchanged in every condition. Environment: the machine of Table 1 with macOS 27.0, PyTorch 2.14.0 and Transformers 5.17.0.

Reference. In inference mode a Transformer has no interaction between samples, so running the batch in one call must give the same outputs as running it in pieces, $f(\text{concat}(X_1, \dots, X_k)) = \text{concat}(f(X_1), \dots, f(X_k))$. We use this metamorphic relation as the reference: the same model on MPS with chunks of 256 texts, whose attention scores have 8.1×10^8 elements. We checked the reference against CPU fp32 on 1000 texts, including the first, middle and last 100, texts 1344–1384 around the predicted boundary, and the texts with the smallest margin between the top two logits. The largest relative logit difference was 6×10^{-5} , and no prediction changed.

Batch sizes. Attention scores have $B \cdot 12 \cdot 512^2$ elements, so $B = 1365$ is just below 2^{32} and $B = 1366$ is just above it. $B = 2048$ gives 1.5×2^{32} . At $L = 512$ the feed-forward activations stay below 2^{32} at all three sizes, so only the attention products cross the boundary. We call a text corrupted when its relative logit error exceeds 10^{-3} .

Table 5: Sentiment classification on MPS (PyTorch 2.14.0, macOS 27.0) with the whole batch in one forward pass, compared with chunked execution. “Scores” is the number of attention-score elements minus 2^{32} . Accuracy is on the three-class TweetEval labels; the value in parentheses is the reference.

B	attention	scores $- 2^{32}$	corrupted texts	changed predictions	max. rel. error	accuracy
1365	eager	−1,048,576	0	0	0	0.708 (0.708)
1366	eager	+2,097,152	1	1	0.78	0.707 (0.708)
1366	SDPA	+2,097,152	0	0	1.9×10^{-5}	0.708 (0.708)
2048	eager	+ 2^{31}	683 (33.4%)	386 (18.9%)	5.16	0.639 (0.713)
2048	SDPA	+ 2^{31}	0	0	1.9×10^{-5}	0.713 (0.713)

Results. Table 5 and Figure 2 show the outcome. Every run finished normally, with no error, warning, NaN or Inf. At $B = 1365$ the single call matches the reference bit for bit. At $B = 1366$ exactly one text, the last one, is corrupted and its prediction changes. At $B = 2048$ texts 1365–2047 are corrupted, 386 predictions (18.9%) change, and accuracy drops from 0.713 to 0.639. Every text before position 1365 still matches the reference bit for bit. Fused SDPA on the same batches agrees with the reference to within 1.9×10^{-5} .

Effect on predictions. At $B = 2048$ the two groups of positions do not overlap: all 1365 texts before position 1365 have a relative error of exactly zero, and all 683 texts from position 1365 onward exceed the threshold, with a

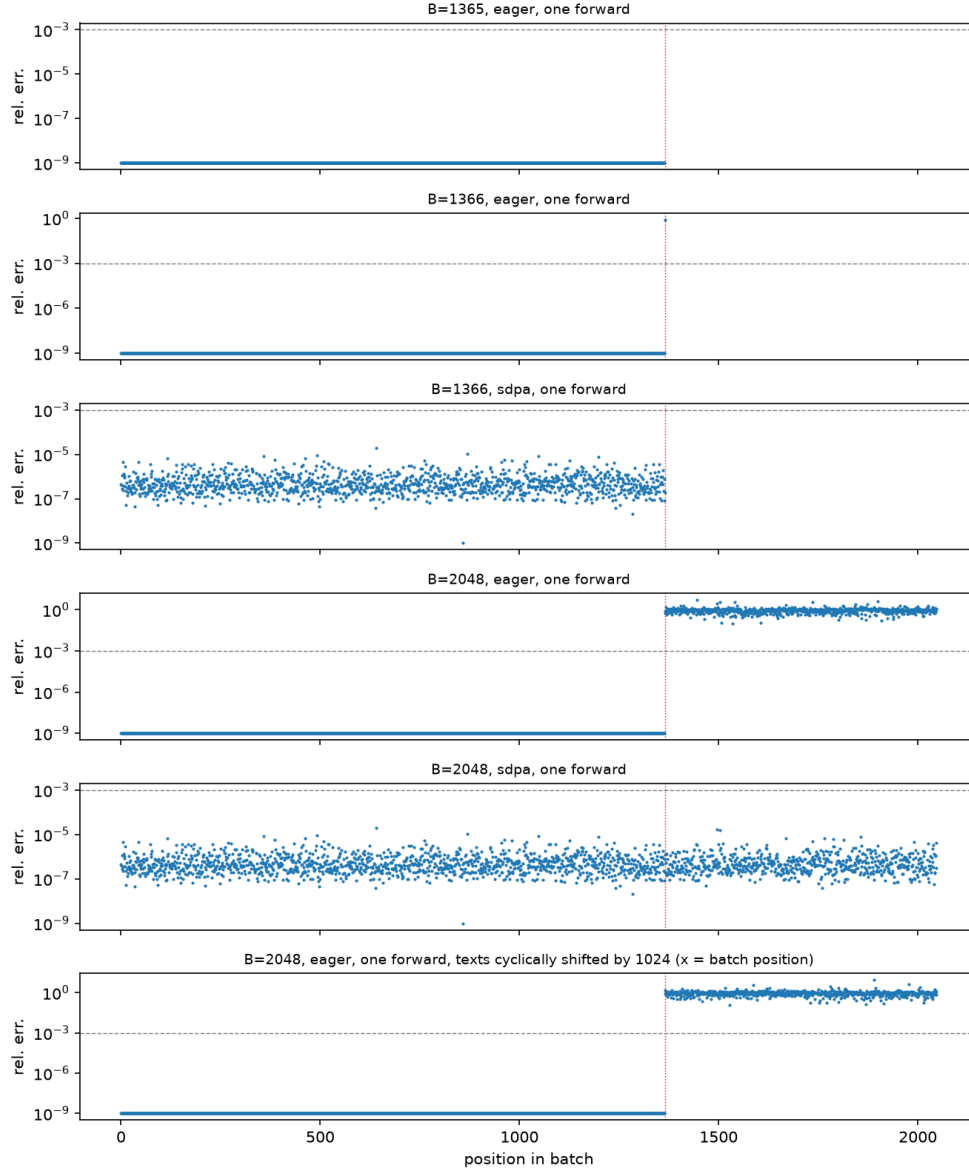


Figure 2: Relative logit error of each text against the chunked reference, by position in the batch. Exact agreement is drawn at 10^{-9} . The dotted red line marks text 1365, the first text predicted to cross 2^{32} score elements; the dashed gray line is the 10^{-3} threshold. The last panel runs the same 2048 texts cyclically shifted by 1024.

minimum of 0.10 and a median of 0.91. The corrupted outputs are not spread over the classes. Of the 683 corrupted texts, 661 are predicted as *neutral*, whereas the reference assigns 242, 297 and 144 texts to negative, neutral and positive. Accuracy on these 683 texts falls from 0.723 to 0.501, which is what a constant *neutral* prediction would score, since 344 of them (0.504) carry that label. The reference is correct and the single call wrong on 266 texts, and the reverse holds on 114 (exact McNemar test, $p = 4.1 \times 10^{-15}$). The damage is therefore much larger per class than overall. Over all 2048 texts, accuracy drops by 7.4 points, while recall drops from 0.788 to 0.529 for negative and from 0.740 to 0.504 for positive, and rises from 0.656 to 0.758 for neutral; on the 683 corrupted texts, recall for negative and positive is 0.010 and 0.054. Studies of nondeterminism in training observe the same pattern, with per-class and subgroup metrics varying far more than overall accuracy [12, 13], although the cause there is rounding-level variation amplified by training and here it is an order-one error in a single forward pass. Because the corrupted outputs collapse onto one class, the visible loss depends on the label distribution, and a data set dominated by that class could hide it.

Where the corruption starts. One head’s score matrix has $512^2 = 2^{18}$ elements, so 2^{32} elements correspond to $2^{14} = 16384 = 1365 \times 12 + 4$ head matrices. If positions past element 2^{32} of the flattened scores are affected, corruption should begin at head 4 of text 1365. Comparing the intermediate tensors of the first attention layer with the reference, the scores $QK^\top$ and the softmax output agree bit for bit at every batch size, and the first corrupted (text, head) pair in the product $\text{softmax}(QK^\top)V$ is (1365, 4), with heads 4–11 of text 1365 corrupted. Feeding the reference softmax output into that product alone reproduces the same corruption. This matches the layer-level result that bmm fails when an input exceeds 2^{32} elements.

Position, not content. We repeated $B = 2048$ with the texts cyclically shifted by 1024. The corrupted batch positions are again 1365–2047, and the corrupted texts (original indices 341–1023) do not overlap with those of the unshifted run. Whether a text is misclassified depends on where it sits in the batch, not on what it says.

A loud failure at $L = 256$. We first tried $L = 256$. There the feed-forward activation $(B \cdot 256) \times 3072$ crosses 2^{32} at the same $B = 5462$ as the scores, and the process aborted inside a Metal matrix-multiplication assertion (destination [3072, 1398272] is too large for kernel), while $B = 5461$ matched the reference exactly. Whether a given oversized operation fails loudly or silently therefore depends on which operation crosses the boundary first.

4.3 Other operations and CUDA

First-pass suite. Before the sweep we ran a suite of 26 test cases on all 11 PyTorch releases, with one fixed shape per case and a comparison against the CPU at sampled batch indices. Table 6 shows the cases that involve matrix multiplication and attention. The bmm rows agree with the sweep, and the suite adds two observations. Eager attention is silently wrong from 2.5.1 onward, as expected from the bmm results, except on 2.13.0, where the process aborts. On 2.4.1 it is correct at this shape, which the bmm rows alone do not explain. Fused SDPA is silently wrong up to 2.12.1 when its scores exceed 2^{32} elements and correct from 2.13.0. A dispatch trace shows that on MPS fused SDPA resolves to `aten::_scaled_dot_product_attention_math_for_mps` in both 2.12.1 and 2.14.0, and not to the FlashAttention operators that PyTorch provides for CUDA.

Table 6: First-pass results on macOS 27.0 by PyTorch version. **W**: silently wrong (relative error against CPU above 10^{-2}); **ok**: correct; **err**: raises; **crash**: process aborts. Output-side cases use $(B, 256, 64) \times (B, 64, 256)$ with $B = 65552$ or 68544 ; the input-side case uses $(B, 256, 256) \times (B, 256, 64)$ with $B = 65552$; attention uses scores of shape $(5712, 12, 256, 256)$.

case	2.4.1	2.5.1	2.6.0	2.7.1	2.8.0	2.9.1	2.10.0	2.11.0	2.12.1	2.13.0	2.14.0
bmm, output $> 2^{32}$, contiguous	W	ok	ok	ok	ok	ok	ok	ok	ok	ok	ok
bmm, output $> 2^{32}$, b transposed	W	W	W	W	W	W	W	W	W	W	W
bmm, output $> 2^{32}$, a transposed	W	W	W	W	W	W	W	W	W	W	W
bmm, input $> 2^{32}$, contiguous	W	W	W	W	W	W	W	W	W	W	W
eager attention	ok	W	W	W	W	W	W	W	W	crash	W
fused SDPA	W	W	W	W	W	W	W	W	W	ok	ok

Regression in 2.14.0. On 2.14.0, `torch.arange` with $N = 2^{32} + 2^{20}$ elements writes only the first $N \bmod 2^{32}$ elements and leaves the rest zero without an error. Versions 2.12.1 and 2.13.0 raise instead (MPSGraph does not support tensor dims larger than INT_MAX). This is the only regression that we found in 2.14.0; for bmm the sweep shows no difference between 2.10.0 and 2.14.0 (Table 4).

CUDA as a control. Correctness in this report is always judged against float64 on the CPU. CUDA serves a different purpose: it shows whether a failure belongs to the operation at that size or to the MPS backend. We ran the first-pass bmm configurations on an NVIDIA A100 (80 GB, PyTorch 2.11.0 with CUDA 12.8, TF32 disabled): output-side and input-side shapes, $B \in \{65520, 65552, 68544\}$, fp32 and fp16, and four layouts (contiguous, either operand transposed, sliced), 48 configurations in total. All 48 are correct. Compared over all elements with float64 on the same GPU, the largest relative error is 9.2×10^{-7} in fp32 and 4.5×10^{-4} in fp16, and spot checks of three batches per configuration against float64 on the CPU agree. Eager attention and fused SDPA with scores of shape $(5712, 12, 256, 256)$ are also correct, with errors below 10^{-6} . The bmm failures are therefore specific to the MPS backend and are not inherent to these shapes. CUDA is not free of problems at this boundary, however. On the same A100, `torch.arange` with $N = 2^{32} + 2^{20}$ int64 elements leaves every element from index 2^{32} onward at zero, without an error, in all four versions we tried (2.11.0 to 2.14.0) [22]. MPS on 2.14.0 fails differently on the same call, writing only the first

$N \bmod 2^{32}$ elements. Two consequences follow. A second GPU backend cannot serve as the oracle for a differential test at this scale, which is why we use the CPU in float64 and, for `arange`, the closed form $x_i = i$. And silent failures at 2^{32} elements are not confined to one backend, which suggests that this size range is rarely exercised by existing tests.

5 Mitigation

One rule. A result that is silently wrong is worse than an error, so the mitigation should turn the first into the second. The rules of Table 3 could in principle be used to let the safe cases through, but Table 4 shows that they change between versions, and Table 6 shows that attention and fused SDPA fail on their own schedule. We therefore use a single rule that refers to neither the version nor the operation: *do not create or read an MPS tensor with 2^{32} or more elements*. Large batches are split into chunks before they reach the device, as in the chunked reference of Section 4.2. `bmm`, and attention with it, has no interaction between batches, so chunking does not change the result.

Guard. The released guard enforces the rule. It is a `TorchDispatchMode` that inspects the inputs and outputs of every aten operation on MPS and raises an exception when one of them has 2^{32} or more elements. For `mm`, `bmm`, `addmm`, `baddbmm` and fused SDPA it estimates the size before the operation runs, from the operand shapes (for SDPA, from the shape of the scores), because some versions abort the process inside the operation. For `bmm` the message states how many batches per chunk keep every operand and the output below the limit. The guard is installed with one call and needs no change to the model code.

Evaluation against the sweep. We applied the stopping rule to the recorded outcome of every run (Table 7). Over all 11 versions, 481 of 5240 runs are silently wrong or return zeros, and the rule stops all of them. The threshold has to be inclusive: with “more than 2^{32} ” the rule would pass 16 runs on 2.4.1 in which a view of exactly 2^{32} elements returns zeros. In no version is a run below 2^{32} elements silently wrong. The price is that the rule also stops 630 of the 4083 correct runs, namely those in which a tensor of 2^{32} or more elements happens to be handled correctly by that version. Run on the first-pass suite, the guard leaves no silent failure and no crash on 2.11.0, 2.12.1 and 2.14.0.

Table 7: The stopping rule applied to the recorded outcomes of the sweep. “Silent” counts runs that are silently wrong or return zeros.

PyTorch	runs	silent runs	silent runs stopped		correct runs stopped
			$n \geq 2^{32}$	$n > 2^{32}$	
2.4.1	360	56	56	40	40 of 304
2.5.1–2.8.0 (each)	374	42	42	42	48 of 282
2.9.1	360	32	32	32	40 of 274
2.10.0–2.13.0 (each)	360	32	32	32	40 of 276
2.14.0 (full sweep)	1584	97	97	97	198 of 1273
all	5240	481	481	465	630 of 4083

Overhead. The guard adds Python work to every operation. On a small training loop (a two-layer network on 1024×120 inputs, 2000 steps) it increases the run time by a factor of 1.73, and on large attention forward passes (batch 64, 32 heads, length 1024, fp16) by a factor of 1.00. The cost is thus negligible in the workload with large tensors, where the guard is needed, and the guard can be left out in workloads whose tensors are small by construction.

6 Conclusion

6.1 Key Findings

- **Silent failures at 2^{32} elements.** On macOS 27.0, `bmm` on the MPS backend returns wrong values without an error when its output exceeds 2^{32} elements and an operand is transposed, and when a contiguous input exceeds 2^{32} elements. Both failures appear in all 11 PyTorch releases from 2.4.1 to 2.14.0, and they are unchanged since 2.5.1. What changed between versions is how loudly an oversized view fails: zeros in 2.4.1, a process abort or a wrong result up to 2.8.0, and an exception from 2.9.1. The limit is on the number of elements and not on bytes, and the result is still correct at exactly 2^{32} elements on 2.14.0.

- **Three rules, and their order.** Three rules reproduce all 1088 outcomes of the sweep with random inputs. Because views with 2^{31} or more elements raise an error only while the output stays within 2^{32} elements, a slightly larger problem can turn an explicit error into a silent failure.
- **What the wrong values are.** In the first failure the whole output is wrong and equals a computation that ignores the strides of the transposed operand. In the second, only the batches beyond element 2^{32} are wrong, and they equal a computation whose index wraps around at 2^{32} . Index-encoded inputs confirm both, element by element.
- **Real workloads are affected, and aggregate metrics hide it.** With eager attention, one oversized batch of a public sentiment classifier corrupts a third of the outputs. The corrupted predictions collapse onto one class, so overall accuracy falls by 7 points while recall for the other two classes falls to almost zero on the affected inputs. Which texts are affected depends on their position in the batch and not on their content.
- **Not only MPS.** The same bmm configurations are correct on CUDA, but `torch.arange` is silently wrong above 2^{32} elements on CUDA as well, in a different way. A second GPU backend is therefore not a safe oracle at this scale.
- **One rule is enough in practice.** Do not create or read an MPS tensor with 2^{32} or more elements, and split large batches before they reach the device. Applied to the recorded outcomes of all 11 versions, this rule stops all 481 silent runs (wrong values or zeros), and no run below 2^{32} elements is silently wrong. The threshold must be inclusive, because 2.4.1 returns zeros for a view of exactly 2^{32} elements.

6.2 Limitations

This first version reports what we could measure on one machine, and several limits follow from that.

Environment. All MPS results come from a single Mac Studio (M2 Ultra) with macOS 27.0. We did not run macOS 14, 15 or 26. That the behavior begins with macOS 15 rests on the error message that PyTorch raised before that release, and not on our own measurements. Other chips and memory sizes are untested. The next version will repeat the sweep on several macOS versions.

Coverage. The systematic sweep covers bmm only. Other operations are covered by the first-pass suite with one shape each, so for them we know that a failure exists, but not where it begins. The sweep uses fp32 and fp16 and does not include bfloat16. Its batch sizes end just above 2^{32} elements; only the case study goes further, to 1.5 times that size. Because of memory, no configuration has an output and an input above 2^{32} elements at the same time. Only forward computation is tested for bmm, so we make no statement about gradients or training. The sweep of earlier PyTorch versions uses a coarser grid, one seed for the full comparisons and the base shapes only.

Interpretation. The rules in Table 3 summarize observed outcomes for one PyTorch version. They agree with every run, but they were fitted after the fact to a finite set of configurations, and they may not hold outside it. The CUDA control covers the first-pass configurations on one PyTorch version and not the full sweep.

Case study and guard. The case study uses one model, one data set and one input length. The collapse of the corrupted predictions onto a single class is an observation on this example, and other models may fail differently. The guard was run on the first-pass suite, and its stopping rule was checked against the recorded outcomes of the sweep; these are the same operations that motivated it. Its overhead was measured on two workloads. We recommend building the chunks before they reach the device, as in the case study; slicing chunks out of a device tensor that already has 2^{32} or more elements creates views with a large storage offset, which the sweep does not cover.

References

- [1] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703*, 2019.
- [2] Junichiro Niimi. [MPS] ‘bmm’ silently returns wrong results when the output exceeds 2^{32} elements and an input is non-contiguous. PyTorch GitHub issue #197636, <https://github.com/pytorch/pytorch/issues/197636>, 2026. Opened 2026-09-19.
- [3] Florian Tambo, Amin Nikanjam, Le An, Foutse Khomh, and Giuliano Antoniol. Silent bugs in deep learning frameworks: an empirical study of Keras and TensorFlow. *Empirical Software Engineering*, 2023.

- [4] Junjie Chen, Yihua Liang, Qingchao Shen, Jiajun Jiang, and Shuochuan Li. Toward understanding deep learning framework bugs. *ACM Transactions on Software Engineering and Methodology*, 2023.
- [5] Gan Wang, Zan Wang, Junjie Chen, Xiang Chen, and Ming Yan. An empirical study on numerical bugs in deep learning programs. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.
- [6] Hung Viet Pham, Thibaud Lutellier, Weizhen Qi, and Lin Tan. CRADLE: Cross-backend validation to detect and localize bugs in deep learning libraries. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019.
- [7] Anjiang Wei, Yinlin Deng, Chenyuan Yang, and Lingming Zhang. Free lunch for testing: Fuzzing deep-learning libraries from open source. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022.
- [8] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. NNSmith: Generating diverse and valid test cases for deep learning compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 2023.
- [9] Jiawei Liu, Yuheng Huang, Zhijie Wang, Lei Ma, Chunrong Fang, Mingzheng Gu, Xufan Zhang, and Zhenyu Chen. Generation-based differential fuzzing for deep learning libraries. *ACM Transactions on Software Engineering and Methodology*, 2023.
- [10] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023.
- [11] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1), 2018.
- [12] Hung Viet Pham, Shangshu Qian, Jiannan Wang, Thibaud Lutellier, Jonathan Rosenthal, Lin Tan, Yaoliang Yu, and Nachiappan Nagappan. Problems and opportunities in training deep learning software systems: An analysis of variance. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020.
- [13] Donglin Zhuang, Xingyao Zhang, Shuaiwen Leon Song, and Sara Hooker. Randomness in neural network training: Characterizing the impact of tooling. *arXiv preprint arXiv:2106.11872*, 2021.
- [14] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. Silent data corruptions at scale. *arXiv preprint arXiv:2102.11245*, 2021.
- [15] Peter H. Hochschild, Paul Turner, Jeffrey C. Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E. Culler, and Amin Vahdat. Cores that don't count. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*, 2021.
- [16] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations (ICLR)*, 2015. arXiv:1409.0473.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30:5998–6008, 2017.
- [18] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.
- [19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [20] Daniel Loureiro, Francesco Barbieri, Leonardo Neves, Luis Espinosa Anke, and Jose Camacho-Collados. TimeLMs: Diachronic language models from Twitter. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 2022.
- [21] Francesco Barbieri, Jose Camacho-Collados, Luis Espinosa Anke, and Leonardo Neves. TweetEval: Unified benchmark and comparative evaluation for tweet classification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.

- [22] Junichiro Niimi. [CUDA] ‘torch.arange’ silently leaves every element from index 2^{32} onward as zero when $N > 2^{32}$. PyTorch GitHub issue #197673, <https://github.com/pytorch/pytorch/issues/197673>, 2026. Opened 2026-09-19.